Term Project

CS 151: Object-Oriented Design

Functional-Spec - v1

Prof: Ahmad Yazdankhah

Team Number	19	**Section**	02
Team Members	1. Sean Reece Calantoc: Use Cases 2. Kanishka Yadav: Use Cases 3. Hoang Khang Pham: 3 Mockups 4. Cheyenne Khouri: Overview, Objective		
Software Name	KnowledgeTrack		

1 Introduction

1.1 Objective

This document is about the process of building a software called KnowledgeTrack to help faculty members have easy access to all the information of each student. This software allows faculty members to search for students either by name or any other characteristic, such as academic status, job status, programming language knowledge, etc. As well as having the ability to edit, create, or delete profiles.

1.2 References

1. Lecture notes and course materials from CS151 (San Jose State University)
2. Yazdankhah, Ahmad, “CS151 Lecture Notes, Fall 2025”
3. Tools used for mockups: Adobe XD, moqups, Microsoft PowerPoint / draw.io / Figma

4. Discussions during lectures on software requirements and use case design

1.3 Acronyms and Abbreviations

SDLC	Software Development Lifecycle
UC	Use Case
FM	Faculty member

2 Software Overview

2.1 Problem Statement

Information about each student is hard to keep track of, information such as academic status, skills, career goals, and behavioral patterns. This information is commonly lost between emails, advising meetings, term projects, etc. As a result, faculty members have a difficult time keeping track of how their student is performing academically and professionally. In order to solve this problem, software such as KnowledgeTrack has been created to minimize the loss of information and to create a well-developed, accessible, long-term profile that faculty members can access whenever the profile of a student is needed.

2.2 Scope

This product is meant to help faculty members easily access each student's information, regardless of their technical background.

2.3 Functional Requirements

1. Faculty can look for a student by searching their name

2. If there are multiple students with the same name, a list is provided for the user to choose from
3. If a faculty member can't find a student through their name, they can use achievements, academic status, or career goals to find them
4. Once the student is selected, a list of achievements, academic status, skills, career goals, and any other information is listed
    a. As well as information such as a current job, job details, programming languages they know
5. There are options to edit or delete the profile
    a. There is a textbox if faculty wishes to add comments over time
6. A checkbox is provided to choose if a student will be whitelisted or blacklisted
7. If a faculty member tries to create a profile for a student under the same name, an error will be shown
8. Faculty can also generate reports in bulk (ex. Listing all students that share a common skill, all students on blacklist or whitelist) or just generate the report of a singular student

2.4 Nonfunctional Requirements

1. It runs on Windows, Mac, and Linux.
2. It requires no prior technical knowledge to use the software
3. It's a desktop application
4. All data will be saved in the flat files or SQLite

3 Use Cases

Use Case Name	UC1: Define Programming Language
Goal	
Allow a faculty member to add a programming language into the system, so that it can be associated with the student's profiles	
Participating Actors	

Faculty Member(FM) : User interacting with the system

System : Application processing the user input

Glossary

FM: Faculty Member
PFE: Primary Flow of Events
AFE: Alternative Flow of Events

Primary Flow of Events

Trigger

FM launches the application, and selects the “Add Programming Language” option

Steps	Actor's Action	System's Response
1	FM selects “Add Language” from the menu	Displays the programming language input
2	FM enters a language into the labeled text box	Captures the user input
3	FM clicks ”save” button	Validates the input
4		If the input is valid, system stores the language in the database
		Then the system updates list of language available
		Displays a message saying "Success"

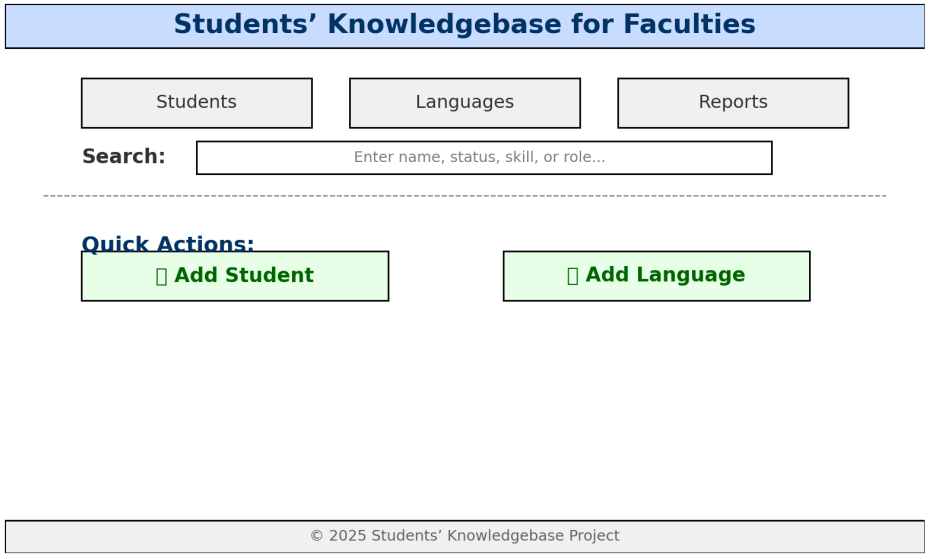
Alternative Flow of Events

Alternative Trigger # 1

FM enters an empty or a language which already exists		
Steps	**Actor's Action**	**System's Response**
1	FM submits empty input	Displays an error message that says “Language name is invalid, please try again”
2	FM enters a language which already exists	Displays an error message that says “Language already exists!”
3	FM edits the input	System waits for a valid input
4	FM resubmits	Returns to step 3 of PFE

4 Mockups

4.1 Home Page



4.2 Define Programming Languages

Students' Knowledgebase for Faculties

Add / Edit Language

Name (required): e.g., Java **Save** Cancel

⚠ Name is required or already exists (case-insensitive).

Existing Languages	Actions
Java	Edit Delete
Python	Edit Delete
C++	Edit Delete
JavaScript	Edit Delete
Go	Edit Delete

© 2025 Students' Knowledgebase Project

4.3 Define a Student Profile

Students' Knowledgebase for Faculties

Basic Information

Full Name: e.g., Hoang Pham
Academic Status: Junior ▼
Job Status: Employed Not Employed
Job Details: Intern at Nvidia

Skills and Interests

Languages: Java, Python
Databases Known: MySQL, MongoDB
Preferred Role: Full-Stack ▼

Faculty Evaluation

Comments: Excellent progress this term... +

Add Comment

Future Services Flags

Whitelist [ ] Blacklist [ ]

Save Cancel

© 2025 Students' Knowledgebase Project